

You might get errors like the ones shown below during the service restart:

```
# service sshd restart
Stopping sshd:
[ OK ]
Starting sshd: Could not load host key: /etc/ssh/ssh_host_
ecdsa_key
Could not load host key: /etc/ssh/ssh_host_ed25519_key

[ OK ]
```

The most probable reason for this is that while installing, the installer renamed some file along with which some keys also got renamed. Re-generating these keys (using the `ssh-keygen -A` command) fixes this error:

```
# ssh-keygen -A
ssh-keygen: generating new host keys: ECDSA ED25519
# service sshd restart
Stopping sshd:
[ OK ]
Starting sshd:
[ OK ]
```

The next step in this setup is to generate the SSH keys which we will use with Google Authenticator:

```
# mkdir .ssh (In the home directory of the user)
# cd .ssh
# ssh-keygen -t rsa -b 2048
# chmod 700 ~/.ssh
# chmod 600 ~/.ssh/keyuser
# cat keyuser.pub >> ~/.ssh/authorized_keys
# chmod 600 ~/.ssh/authorized_keys
```

 **Note:** As mentioned before, make sure to generate the keys with the user with which you want to log onto the server. Also, accessing your server using the 'root' user is not recommended at all. I am using 'root' here just for illustration purposes.

Edit the openSSH configuration file again and set the following:

```
PermitRootLogin no
PasswordAuthentication no
UsePAM yes
AuthenticationMethods publickey,keyboard-interactive
```

 **Note:** As per the `AuthenticationMethods` directive, public/private key pairs will be checked first. Once successful, the Google authentication verification code will be asked by the server.

Copy the private keys (key user) to the machine. This will be used to gain access to your server along with Google authentication.

Edit the `/etc/pam.d/sshd` file once again and make the following changes:

```
##PAM-1.0
auth      required      pam_sepermit.so
#auth      include      password-auth
auth      required      pam_google_authenticator.so
account    required      pam_nologin.so
account    include      password-auth
password    include      password-auth
...
...
session    include      password-auth
```

 **Note:** `Password-auth` is now commented out, which means it will not be used in the authentication mechanism.

Restart the openSSH service once again and, hopefully, everything will be in place. The next time you try to log into your server, your keys will be used along with TOTP from the Google Authenticator installed on your phone, in order to provide access to your server.

Using all the above authentication methods (for those who're paranoid)

For the people who are extra careful when it comes to the security of their servers, let's use three-way authentication: publickey, google-authentication and user password.

Uncomment the following line from `/etc/pam.d/sshd` in order to achieve this:

```
#auth      include      password-auth
```

Restart the openSSH service and you will have three-way authentication on your server.

In my opinion, this is not necessary because SSH keys with Google authentication provide a pretty tight and secured login mechanism by themselves. However, if a systems administrator is concerned about such a configuration on multiple servers – for instance, 1000+ machines in a network – then using a third party application such as *jumpcloud* would be much easier to configure and maintain for implementing two-factor authentication across the network.

Today, it is very important to secure your network/infrastructure and, luckily, we have open source tools available to us which we can make use of to achieve that perfect, secured network. 

 **By: Aman Hanjrah**

The author is a Linux administrator/operations engineer and an open source enthusiast. He currently works at Kazan Networks Corp. as the IT head, and is located in Roseville, California (USA).